

P2PChat

Техническое задание на доведение проекта до полноценного релиза

Версия документа: 1.0 • Дата: 30 сентября 2026

Целевая концепция: Discord-подобное приложение без постоянного центрального сервера, где участники комнаты совместно предоставляют сетевую инфраструктуру, а роль координатора автоматически мигрирует между ними.

Исходная база

ТЗ подготовлено с учётом публичной структуры репозитория ASMAXI/p2pchat: 212 коммитов, npm workspace, TypeScript, каталоги lib, artifacts и scripts, CI-конфигурация и MIT-лицензия. На момент подготовки GitHub не предоставляет через страницу репозитория полное содержимое внутренних модулей, поэтому конкретные имена существующих классов/функций не фиксируются как уже реализованные.

1. Цель проекта

Создать рабочее P2P-приложение для частных групповых коммуникаций, в котором комната не зависит от постоянно работающего облачного сервера. Любой участник может стать текущим coordinator/host; при его отключении оставшиеся участники автоматически выбирают нового coordinator и продолжают работу.

Главное отличие проекта

Не просто «чат через WebSocket между пользователями», а распределённая комната с миграцией coordinator, репликацией состояния, обнаружением участников, восстановлением соединений и криптографической идентичностью.

2. Scope релиза 1.0

- Создание комнаты и подключение по invite link/code.
- Постоянная криптографическая identity пользователя.
- Групповой текстовый чат в реальном времени.
- Список участников и статусы online/offline/reconnecting.
- Coordinator election и автоматическая миграция роли.
- Репликация сообщений и критического состояния между peers.
- Reconnect после временного обрыва.
- Защищённый транспорт и end-to-end encryption для содержимого.
- NAT traversal с прямым соединением как приоритетом; relay — только как fallback.
- Локальное хранение истории и восстановление состояния.
- Базовые роли/permissions: owner/admin/member.
- Устойчивость к duplicate/out-of-order events.
- Логи, диагностика, тесты, CI и release build.

Не входит в обязательный 1.0

- Публичная социальная сеть и поиск пользователей.
- Масштабирование до тысяч участников в одной комнате.
- Мобильные клиенты.
- Стриминг экрана и полноценная видеоконференция.
- Глобальная облачная история.
- Обязательная работа без какого-либо инфраструктурного fallback при невозможности прямого P2P-соединения.

3. Целевая архитектура

Логическая схема

Client → Identity → Room → Membership → Coordinator → Replication → Transport → Storage/UI

Каждый экземпляр приложения должен уметь одновременно быть peer и временным coordinator. Coordinator отвечает за координацию протокола, но не должен быть единственной точкой хранения истории.

3.1. Слои

Слой	Ответственность	Ключевое требование
Identity	Ключи, peer ID, подписи	Стабильная identity между запусками

Слой	Ответственность	Ключевое требование
Room	Room ID, metadata, lifecycle	Детерминированное состояние
Membership	Участники, heartbeats, presence	Корректное обнаружение ухода
Coordinator	Election, epoch, coordination	Нет split-brain
Replication	Events, ordering, deduplication	Идемпотентность
Transport	Direct P2P, NAT, relay	Автоматический fallback
Crypto	Key exchange, encryption	Содержимое не раскрывается relay
Storage	History, identity, cache	Атомарность и восстановление
UI	Chat, members, settings	Состояние сети понятно пользователю

4. Identity и безопасность

Каждый пользователь получает долгоживущую криптографическую identity. Display name является изменяемым атрибутом и не должен использоваться как идентификатор.

Минимальная модель

```
PeerIdentity = publicKey + privateKey + peerId + displayName
```

peerId должен быть детерминированно связан с public key. Все coordinator/membership-critical операции должны быть подписаны. Private key хранится локально и не передаётся peer'ам.

- Использовать проверенные криптографические библиотеки, не реализовывать primitives самостоятельно.
- Для identity — современная схема на базе Ed25519 или эквивалентной проверенной библиотеки.
- Для установления сессионных ключей — X25519 или эквивалентный современный механизм.
- Для сообщений/каналов — AEAD (например AES-GCM или ChaCha20-Poly1305).
- Каждый peer проверяет peer ID/public key до принятия подписанного события.
- Invite должен содержать минимально необходимую информацию и не включать private key.
- Relay не должен получать plaintext сообщений.

5. Room protocol

Каждая комната имеет неизменяемый roomId и монотонный epoch coordinator state.

```
RoomState: roomId, epoch, coordinatorPeerId, members, roles, channels, replication cursor.
```

Epoch является защитой от устаревшего coordinator. Сообщения/команды от старого epoch не должны менять текущее authoritative состояние.

5.1. Жизненный цикл

- CREATE_ROOM: создатель генерирует roomId и первоначальное состояние.
- JOIN_REQUEST → JOIN_ACCEPT: новый peer получает snapshot и актуальный epoch.
- SYNC: peer догоняет события после snapshot.
- READY: peer подтверждает, что локальное состояние синхронизировано.
- LEAVE/PEER_TIMEOUT: membership обновляется.
- COORDINATOR_FAILURE: начинается election.
- NEW_EPOCH: победитель публикует новый epoch.
- STATE_RECONCILIATION: участники сверяют snapshot/cursor.

6. Coordinator election

Election должен быть детерминированным и не зависеть от субъективного выбора UI. Для первой версии допускается упрощённый leader election с epoch и majority/ack подтверждением.

Рекомендуемая модель

- Coordinator регулярно публикует heartbeat с текущим epoch.
- Peer считает coordinator недоступным после configurable timeout и нескольких пропущенных heartbeat.
- Запускается election timeout с jitter, чтобы снизить вероятность одновременной кандидатуры всех peers.
- Candidate увеличивает epoch и объявляет кандидатуру.
- Участники голосуют не более одного раза за epoch.
- Победитель публикует COORDINATOR_COMMIT.
- Старые epoch считаются stale.
- При отсутствии quorum комната переходит в RECOVERING, а не создаёт два authoritative coordinator.

7. Репликация состояния

Не хранить историю только у coordinator. Все критические изменения представлены событиями.

```
Event: eventId, roomId, epoch, authorPeerId, sequence, type, timestamp, payload, signature.
```

Требования

- eventId уникален и используется для deduplication.
- Повторное получение одного eventId не меняет состояние второй раз.
- Out-of-order события помещаются в pending buffer до появления зависимостей.
- Критические события имеют predecessor/causal cursor.
- История может периодически compact-иться в snapshot.
- Новому peer передаётся snapshot + хвост событий.
- После coordinator migration состояние сверяется по hash/snapshot digest.

8. Transport и NAT

Приоритет: direct connection. Архитектура не должна предполагать, что два произвольных устройства всегда могут установить прямой канал через Internet.

Порядок подключения

- LAN/direct address.
- IPv6 direct, если доступен.
- UDP hole punching / ICE-подобная схема при наличии signaling/bootstrap.
- Peer relay через уже подключённого участника.
- Опциональный TURN/relay как последний fallback.

Важно: discovery/signaling/relay — разные функции. Даже если для первоначального handshake понадобится внешний сервис, это не делает room state централизованным.

9. Text chat

- Отправка, получение и локальное отображение сообщений.
- Reply/edit/delete — только после базового чата и при наличии корректной event-модели.
- Message ID и author identity обязательны.

- Delivery state: local → sent → acknowledged → synced.
- Offline queue с повторной отправкой.
- Ограничение размера сообщения и rate limit.
- Нормализация Unicode и безопасное отображение пользовательского текста.

10. Voice — Phase 2 после стабильного text core

Голос не должен блокировать завершение базового релиза. Для voice использовать отдельный media layer; текстовый protocol не должен смешиваться с RTP/media traffic.

- Opus.
- Push-to-talk и mute.
- Per-peer connection quality.
- Coordinator/relay для media только при необходимости.
- Автоматическое переключение coordinator для voice session.
- Echo cancellation/AGC/denoise на уровне поддерживаемого runtime.

11. File transfer — Phase 2

- Chunked transfer.
- Hash файла до/после передачи.
- Resume interrupted transfer.
- Прогресс и отмена.
- Опциональная репликация chunks.
- Размеры и типы файлов ограничиваются настройками комнаты.

12. Persistence

- Identity хранится отдельно от room cache.
- Room cache содержит snapshot, event log и metadata.
- Запись должна быть atomic: temp → fsync/commit → rename.
- При старте выполняется integrity check.
- Повреждённый cache не должен ломать запуск приложения.
- Пользователь может удалить локальную историю без удаления identity.

13. UI/UX

- Create Room.
- Join Room по ссылке/коду.
- Connection status.
- Coordinator indicator без акцента на технических деталях.
- Member list.
- Message composer.
- Reconnecting state.
- Clear error when P2P path unavailable.
- Settings: identity, network, storage, privacy, diagnostics.

Пользователь не должен видеть внутренние election/epoch ошибки как необъяснимые сообщения. Например: «Переподключаемся к комнате...», а не «epoch conflict».

14. Protocol message types

Type	Назначение
HELLO	Первичный handshake
AUTH	Подтверждение identity
ROOM_JOIN	Запрос входа
ROOM_SNAPSHOT	Снимок состояния
SYNC_REQUEST	Запрос недостающих событий
SYNC_EVENTS	Передача событий
HEARTBEAT	Контроль coordinator/peer
ELECTION	Кандидатура
VOTE	Голос
COORDINATOR_COMMIT	Фиксация нового coordinator
CHAT_MESSAGE	Сообщение чата
ACK	Подтверждение доставки/синхронизации
LEAVE	Корректный выход
ERROR	Протокольная ошибка

15. Failure scenarios

Сценарий	Ожидаемое поведение
Coordinator выключился	Election → новый epoch → новый coordinator
Peer временно потерял сеть	RECONNECTING → resync после возврата
Два peer считают себя coordinator	Epoch/quorum отвергает stale coordinator
Сообщение пришло дважды	Deduplication по eventId
События пришли не по порядку	Pending buffer + последующая обработка
Новый peer подключился поздно	Snapshot + event tail
Coordinator вернулся	Он становится обычным peer до нового election
Все peers вышли	Комната считается завершённой локально
Cache повреждён	Безопасное восстановление/очистка cache
Direct P2P невозможен	Relay fallback или понятная ошибка

16. Производительность и лимиты 1.0

- Целевой размер комнаты: 2–25 участников.
- Рекомендуемый full-mesh режим: до 8–10 peers; далее — coordinator/relay overlay.
- UI не должен блокироваться сетевыми операциями.
- Все тяжёлые операции — async/non-blocking.
- Backpressure обязателен для больших payload.
- Message/event queues должны иметь ограничение памяти.

17. Testing

- Unit tests: identity, event IDs, ordering, deduplication, epoch validation.
- Integration tests: 2/3/5/10 peers.
- Chaos tests: kill coordinator, kill random peer, network pause/resume.
- Property-based tests для event replication.
- Protocol compatibility tests.
- Persistence crash tests.
- Security tests: forged event, replayed event, stale epoch, unauthorized command.
- E2E smoke test на чистой машине.

Критический E2E-сценарий

А создаёт комнату → В/С подключаются → А отправляет 10 сообщений → А аварийно завершается → В становится coordinator → С остаётся в комнате → А запускается снова → А получает актуальный snapshot и продолжает чат без дубликатов.

18. CI/CD и release

- `pnpm install --frozen-lockfile`.
- `typecheck`.
- lint/format check.
- unit tests.
- integration tests.
- build всех артефактов.
- Smoke launch.
- Release artifacts с version metadata.
- Changelog.

Текущий root package уже содержит build/typecheck workflow на `pnpm` и TypeScript, поэтому эти проверки следует сохранить и расширять, а не заменять.

19. Definition of Done — релиз 1.0

Область	Готово, если...
Room	Создание/вход/выход работают повторяемо
Identity	Пeer сохраняет identity между перезапусками
Chat	Сообщения доставляются и не дублируются
Replication	Новый peer восстанавливает состояние
Election	Coordinator migration проходит автоматически
Security	Нельзя принять неподписанное критическое событие
Reconnect	Временный сетевой сбой восстанавливается
NAT	Direct/relay strategy понятна и тестируется
Persistence	Crash не уничтожает валидное состояние
Testing	Критический E2E сценарий проходит автоматически
Release	Чистая установка собирается и запускается
Docs	README содержит архитектуру, запуск и ограничения

20. Архитектурные принципы, которые нельзя нарушать

- Coordinator — временная роль, а не источник истины.
- Никаких plaintext секретов в логах.
- Никаких самописных криптографических primitives.
- Никаких silent data loss при coordinator migration.
- Все distributed events должны быть идемпотентны.
- UI не должен зависеть от конкретного transport implementation.
- Network failures считаются нормальным состоянием, а не исключением.
- Каждое изменение протокола получает version/compatibility policy.

21. Рекомендуемая структура модулей

lib/ identity/ # keys, peerId, signatures crypto/ # sessions, AEAD, key management protocol/ # wire messages, schemas, versioning transport/ # direct, NAT, relay discovery/ # bootstrap/discovery room/ # room lifecycle and state machine membership/ # peers, heartbeat, presence coordinator/ # election, epoch, quorum replication/ # events, snapshots, sync storage/ # local persistence chat/ # messages and delivery media/ # voice/files (phase 2) diagnostics/ # logs, metrics, debug tools artifacts/ desktop/ # user-facing application scripts/ build/test/release tooling

22. Приоритет реализации

P0: protocol + identity + room state + transport abstraction.

P1: membership + replication + coordinator election.

P2: text chat + persistence + reconnect.

P3: NAT/relay hardening + security audit + chaos tests.

P4: UX polish + packaging + release.

P5: voice/files.

Итог: сначала сделать безупречным распределённое ядро. Функции Discord поверх него добавляются позже.